

Thread states

Just like processes, threads have states.

Running → the thread is actively executing on a CPU.

Ready → the thread is prepared to run, but waiting for CPU scheduling.

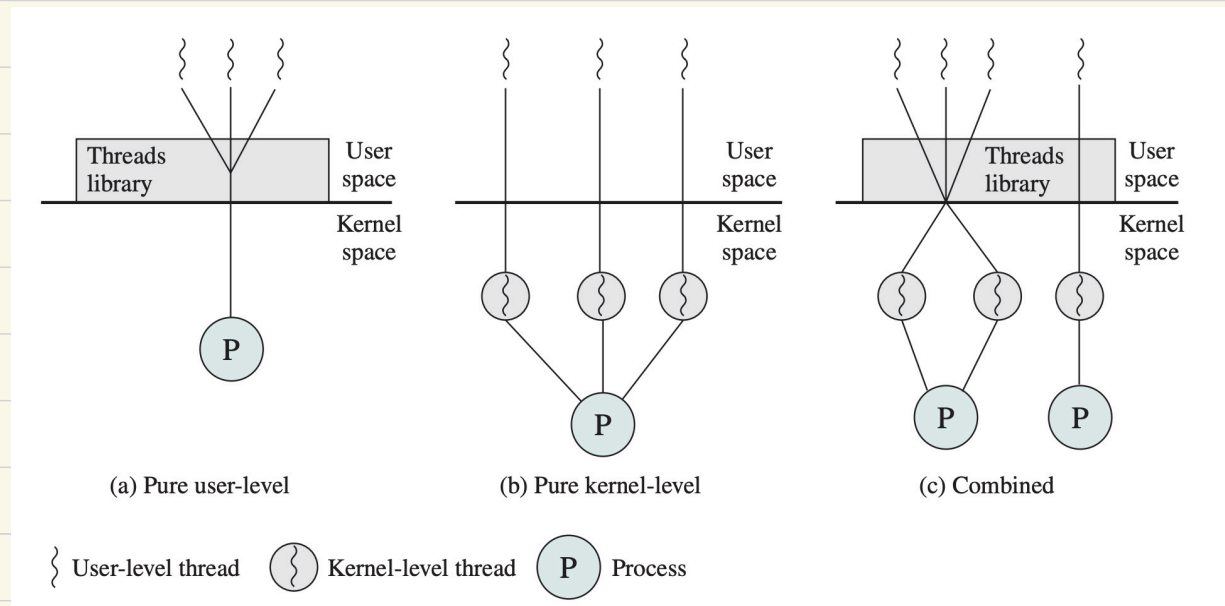
Blocked/waiting → the thread cannot proceed until some event happens (eg: I/O finishes)

These states are tracked by the **Thread control block (TCB)**

Types of threads

* User-level threads (ULT)

- thread management done by the application and the kernel is not aware of the existence of threads.
(use thread library)



- thread library manages the scheduling/management of threads within the process.
- Here, the kernel continues to schedule the process as a unit and assigns a single execution state (Ready/Running/Blocked) to that process. → if one U/T makes a blocking system call, the entire process (and thus all threads) are blocked.

Advantages:

* Very fast to create/switch threads (no kernel intervention)
no overhead of mode switches while switching.

* Scheduling can be application specific

like RR scheduling algo for one application, while another might benefit from a priority-based algo.

→ can tailor to the application without disturbing the underlying OS scheduler.

Disadvantages:

* Poor concurrency if blocking calls are frequent.

* a multi-threaded application cannot take advantage of multi-processing.

* Kernel-level Threads (KLT)

- all the work of thread management is done by the kernel.
(Use thread API → which we will see in detail later!)
- kernel maintains context information for the process as a whole and individual threads within the process.
- scheduling by the kernel is done on a thread basis.

Advantages

- * Overcome the disadvantages of ULT approach.
- * Kernel routines themselves can be multi-threaded.

Disadvantages

- * Transfer of control from one thread to another within the same process requires a mode switch to kernel.

Combined approaches

- the multiple ULTs from a single application are mapped onto some (smaller or equal) number of KLTs.

- multiple threads within the same application can run in parallel on multiple processors, and a blocking system call need not block the entire process.

eg:- Solaris

An example: Thread creation

A program that creates two threads, one printing "A" and other "B"

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4
5  #include "common.h"
6  #include "common_threads.h"
7
8  void *mythread(void *arg) {
9      printf("%s\n", (char *) arg);
10     return NULL;
11 }
12
13 int main(int argc, char *argv[]) {
14     if (argc != 1) {
15         fprintf(stderr, "usage: main\n");
16         exit(1);
17     }
18
19     pthread_t p1, p2;
20     printf("main: begin\n");
21     Pthread_create(&p1, NULL, mythread, "A");
22     Pthread_create(&p2, NULL, mythread, "B");
23     // join waits for the threads to finish
24     Pthread_join(p1, NULL);
25     Pthread_join(p2, NULL);
26     printf("main: end\n");
27     return 0;
28 }
```

Remark: How exactly a thread is created will be discussed in detail in Thread API (next lecture)

Creates two threads, of which will run the function mythread() with different arguments A and B

Remark: Note that in a procedure call, when you call a function,

caller → callee runs → return → caller continues

But, here `main()` immediately call `pthread_join(p1, ...)` right after creating threads, before they have necessarily been scheduled.

The OS could delay the execution of threads, then `main` gets blocked, waiting for threads to eventually run and finish

* A few possible execution ordering of the program:

(the scheduler decides what to run at a given point).

main	Thread 1	Thread2
starts running		
prints "main: begin"		
creates Thread 1		
creates Thread 2		
waits for T1		
	runs	
	prints "A"	
	returns	
waits for T2		
		runs
		prints "B"
		returns
prints "main: end"		

main	Thread 1	Thread2
starts running		
prints "main: begin"		
creates Thread 1		
	runs	
	prints "A"	
	returns	
creates Thread 2		
		runs
		prints "B"
		returns
waits for T1		
returns immediately; T1 is done		
waits for T2		
returns immediately; T2 is done		
prints "main: end"		

what happens when threads access shared data?

An example:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "common.h"
4  #include "common_threads.h"
5
6  volatile int counter = 0;
7  int loops;
8
9  void *worker(void *arg) {
10     int i;
11     for (i = 0; i < loops; i++) {
12         counter++;
13     }
14     return NULL;
15 }
16
17 int main(int argc, char *argv[]) {
18     if (argc != 2) {
19         fprintf(stderr, "usage: threads <loops>\n");
20         exit(1);
21     }
22     loops = atoi(argv[1]);
23     pthread_t p1, p2;
24     printf("Initial value : %d\n", counter);
25     Pthread_create(&p1, NULL, worker, NULL);
26     Pthread_create(&p2, NULL, worker, NULL);
27     Pthread_join(p1, NULL);
28     Pthread_join(p2, NULL);
29     printf("Final value  : %d\n", counter);
30     return 0;
31 }
```

What does each worker Thread
is trying to do?

add a number to the
shared variable counter,
and do so "loops" many times.

- If loops = 10,000 then
the desired result = 20,000 !

unfortunately, following are the results of the program,

Run 1:

```
[dalujacob@Dalus-MacBook-Pro intro % ./threads 10000  
Initial value : 0  
Final value   : 15164
```

Run 2:

```
[dalujacob@Dalus-MacBook-Pro intro % ./threads 10000  
Initial value : 0  
Final value   : 18250
```

not only is each run wrong, but also yields a different result!

why did this happen? will see soon!

Race condition (or data race) → when the result of a program depend on the timing of code's execution.

Before explaining the situation in detail, let us look at some simpler examples.

① Suppose we run a program with two threads that do the following

Thread A

$x=1$;

Thread B

$x=2$;

What are the possible final values of x ?

- either $x=1$ or $x=2$ depending on which Thread wins or loses the "race" to set the value of x .

② Suppose that initially $y=12$ and we run a program with two threads that do the following.

Thread A

$x = y + 1;$

Thread B

$y = y * 2$

What are the possible final values of x ?

Thread A executes first $\Rightarrow x=13$

Thread B executes first $\Rightarrow x=25$

③ Suppose that initially $x=0$ and we run a program with two threads that do the following.

Thread A

$x = x + 1;$

Thread B

$x = x + 2;$

What are the possible final values of x ?

Obviously, one possible outcome is $x=3$.

when Thread A runs to completion and then Thread B starts and runs to completion (or vice versa)

Can we get $x=2$ or $x=1$? YES!

But why?

When we write a single statement $x=x+1$, compilers may produce multiple instructions such as,

- ① load memory location x into a register.
- ② add 1 to that register
- ③ store the result to memory location x

So, any of the following scenarios could happen while execution!

Thread A	Thread B
load r1, x	
add r2, r1, 1	
store x, r2	
	load r1, x
	add r2, r1, 2
	store x, r2
final: x == 3	

Thread A	Thread B
load r1, x	
	load r1, x
add r2, r1, 1	
	add r2, r1, 2
store x, r2	
	store x, r2
final: x == 2	

Thread A	Thread B
load r1, x	
	load r1, x
add r2, r1, 1	
	add r2, r1, 2
	store x, r2
store x, r2	
final: x == 1	

The first example (counter) we began with go through a similar scenario.

Recall that in each thread, we add a number 1 to the counter. The corresponding code sequence in x86 may look like follows:

location of the counter

```

100 mov    0x8049a1c, %eax
105 add    $0x1, %eax
108 mov    %eax, 0x8049a1c

```


The race condition can be demonstrated as follows:

a piece of code that access a shared variable

OS	Thread 1	Thread 2	(after instruction)		
			PC	eax	counter
	before critical section		100	0	50
	mov 8049a1c,%eax		105	50	50
	add \$0x1,%eax		108	51	50
interrupt	save T1				
	restore T2				
		mov 8049a1c,%eax	100	0	50
		add \$0x1,%eax	105	50	50
		mov %eax,8049a1c	108	51	50
interrupt	save T2		113	51	51
	restore T1				
	mov %eax,8049a1c		108	51	51
			113	51	51

what we need to solve this problem?

Mutual exclusion → a guarantee that if one thread is executing within the critical section, the others will be prevented from doing so.

The above terms are coined by Edsger Dijkstra (pioneer in the field and won the Turing Award for this and related works)

Idea: make a series of actions "atomic"
i.e., "all or nothing"

When the instruction is executed, it would perform the update as desired (i.e., it could not be interrupted mid-instruction)

↳ How to do this? → will see in chapter for locks!

One more problem: what to do when one thread must wait for another to complete some action before it continues?

↳ How to perform such interactions?

→ will see in chapter for condition variables.